

Asignatura	Datos del alumno	Fecha
Bases de Datos Avanzadas	Apellidos: Herrera Ramírez	Mayo 17 de 2026
	Nombre: Andres Mateo	

Título laboratorio: Almacenamiento y validación de ficheros XML en Oracle

Nombre del autor: Andrés Mateo Herrera Ramírez

Nombre de la institución: Fundación Universitaria Internacional de La
Rioja-UNIR

Nombre del curso: Bases de Datos Avanzadas

Nombre del profesor: Deivis Eduard Ramírez Martínez

17 de Mayo de 2026

Asignatura	Datos del alumno	Fecha
Bases de Datos Avanzadas	Apellidos: Herrera Ramírez	Mayo 17 de 2026
	Nombre: Andres Mateo	

Introducción: En la actualidad, el intercambio de información entre aplicaciones y sistemas distribuidos requiere formatos estructurados, flexibles y fácilmente interpretables tanto por máquinas como por usuarios. XML (eXtensible Markup Language) se ha consolidado como uno de los estándares más utilizados para la representación y transferencia de datos debido a su capacidad de describir información de manera jerárquica y extensible.

En el presente laboratorio se abordaron los principales mecanismos relacionados con el almacenamiento, generación y validación de documentos XML dentro de un sistema gestor de bases de datos Oracle. Para ello, se trabajó con el esquema HR proporcionado por Oracle, a partir del cual se recuperó información relacional y se transformó a formato XML utilizando funcionalidades SQL/XML.

Adicionalmente, se desarrolló un esquema XML (XSD) con el propósito de definir la estructura y restricciones de los documentos generados, permitiendo posteriormente validar la integridad y consistencia de los datos obtenidos. También se realizaron pruebas de inserción de documentos válidos e inválidos en el sistema gestor de bases de datos, con el fin de analizar el comportamiento del mecanismo de validación XML implementado por Oracle.

Finalmente, este laboratorio permitió comprender la integración entre bases de datos relacionales y tecnologías XML, así como la importancia de los estándares de validación y consulta en entornos modernos de almacenamiento e intercambio de información.

Objetivos:

Objetivo general: Implementar mecanismos de almacenamiento y validación de documentos XML en Oracle utilizando SQL/XML y esquemas XML

Objetivos específicos:

- Generar información XML a partir del esquema HR
- Crear un esquema XML (XSD)
- Validar documentos XML

Asignatura	Datos del alumno	Fecha
Bases de Datos Avanzadas	Apellidos: Herrera Ramírez	Mayo 17 de 2026
	Nombre: Andres Mateo	

- Registrar esquemas XML en Oracle
- Realizar inserciones válidas e inválidas

Herramientas y entorno de trabajo:

- Oracle 11g
- SQL Developer
- Notepad++
- Windows 11
- GitHub
- Herramienta de validación XML

Desarrollo del laboratorio: Mecanismo por el cual se generaron, almacenaron y validaron documentos XML utilizando Oracle Database y las funcionalidades proporcionadas por SQL/XML. Se trabajó con el esquema HR de Oracle, a partir del cual se recuperó información relacional para posteriormente transformarla a formato XML. Asimismo, se diseñó un esquema XML (XSD) con el propósito de definir la estructura y restricciones del documento generado, permitiendo validar la integridad de la información almacenada.

- Diseño del esquema XML (XSD)
- Generación del documento XML utilizando SQL/XML
- Validación del documento XML
- Registro del esquema XML en Oracle
- Inserción de documentos válidos e inválidos en la base de datos
- Análisis de resultados obtenidos

Asignatura	Datos del alumno	Fecha
Bases de Datos Avanzadas	Apellidos: Herrera Ramírez	Mayo 17 de 2026
	Nombre: Andres Mateo	

Memoria explicativa:

Ilustración 1 Esquema XSD

```

1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
4
5   <!-- Elemento raíz -->
6   <xs:element name="Empleados">
7
8     <xs:complexType>
9       <xs:sequence>
10
11         <!-- Lista de empleados -->
12         <xs:element name="Empleado" maxOccurs="unbounded">
13
14           <xs:complexType>
15
16             <xs:sequence>
17
18               <xs:element name="Nombre" type="xs:string"/>
19               <xs:element name="Apellido" type="xs:string"/>
20               <xs:element name="Correo" type="xs:string"/>
21               <xs:element name="Salario" type="xs:decimal"/>
22               <xs:element name="Departamento"
23                 type="xs:string"/>
24             </xs:sequence>
25
26             <!-- Atributo ID -->
27             <xs:attribute name="id" type="xs:integer"
28               use="required"/>
29           </xs:complexType>
30         </xs:element>
31       </xs:sequence>
32     </xs:complexType>
33   </xs:element>
34 </xs:schema>

```

Se diseñó un esquema XSD que define la estructura del documento XML a generar. El elemento raíz es <Empleados>, el cual contiene uno o múltiples elementos <Empleado>. Cada empleado lleva un atributo obligatorio id de tipo entero, y cinco elementos hijos: Nombre, Apellido, Correo, Salario y Departamento. El atributo id se marcó como use="required" para garantizar su presencia en todo documento válido.

Asignatura	Datos del alumno	Fecha
Bases de Datos Avanzadas	Apellidos: Herrera Ramírez	Mayo 17 de 2026
	Nombre: Andres Mateo	

Ilustración 2 Consulta SQL / XML básica (una sola tabla)

The screenshot shows the SQL Developer interface. The left pane displays the 'CONNECTIONS' and 'SQL SNIPPETS' sections. The main editor window contains the following SQL query:

```

1 SELECT xmlelement(
2     "Empleado",
3     xmlforest(
4         employee_id AS "ID",
5         first_name AS "Nombre",
6         last_name AS "Apellido",
7         salary AS "Salario"
8     )
9 ) AS resultado_xml
10 FROM hr.employees;

```

The 'QUERY RESULT' tab shows the output of the query, which is an XML document. The first few rows of the result are:

```

1 <Empleado><ID>100</ID><Nombre>Steven</Nombre><Apellido>King</Apellido><Sa_
2 <Empleado><ID>101</ID><Nombre>Neena</Nombre><Apellido>Yang</Apellido><Sa_
3 <Empleado><ID>102</ID><Nombre>Lex</Nombre><Apellido>Garcia</Apellido><Sa_
4 <Empleado><ID>103</ID><Nombre>Alexander</Nombre><Apellido>James</Apellid_
5 <Empleado><ID>104</ID><Nombre>Bruce</Nombre><Apellido>Miller</Apellido><_
6 <Empleado><ID>105</ID><Nombre>David</Nombre><Apellido>Williams</Apellido_
7 <Empleado><ID>106</ID><Nombre>Valli</Nombre><Apellido>Jackson</Apellido_
8 <Empleado><ID>107</ID><Nombre>Diana</Nombre><Apellido>Nguyen</Apellido_
9 <Empleado><ID>108</ID><Nombre>Nancy</Nombre><Apellido>Gruenberg</Apellid_
10 <Empleado><ID>109</ID><Nombre>Daniel</Nombre><Apellido>Faviet</Apellido_
11 <Empleado><ID>110</ID><Nombre>John</Nombre><Apellido>Chen</Apellido><Sal_
12 <Empleado><ID>111</ID><Nombre>Ismael</Nombre><Apellido>Sciarra</Apellido_
13 <Empleado><ID>112</ID><Nombre>Jose Manuel</Nombre><Apellido>Uman</Apell_
14 <Empleado><ID>113</ID><Nombre>Luis</Nombre><Apellido>Popp</Apellido><Sal_
15 <Empleado><ID>114</ID><Nombre>Den</Nombre><Apellido>Li</Apellido><Salari_
16 <Empleado><ID>115</ID><Nombre>Alexander</Nombre><Apellido>Khoo</Apellido_
17 <Empleado><ID>116</ID><Nombre>Shelli</Nombre><Apellido>Baida</Apellido><_
18 <Empleado><ID>117</ID><Nombre>Sigal</Nombre><Apellido>Tobias</Apellido><_
19 <Empleado><ID>118</ID><Nombre>Guy</Nombre><Apellido>Himuro</Apellido><Sa_
20 <Empleado><ID>119</ID><Nombre>Karen</Nombre><Apellido>Colmenares</Apelli_
21 <Empleado><ID>120</ID><Nombre>Matthew</Nombre><Apellido>Weiss</Apellido_
22 <Empleado><ID>121</ID><Nombre>Adam</Nombre><Apellido>Fripp</Apellido><Sa_
23 <Empleado><ID>122</ID><Nombre>Payam</Nombre><Apellido>Kaufling</Apellido_
24 <Empleado><ID>123</ID><Nombre>Shanta</Nombre><Apellido>Vollman</Apellido_
25 <Empleado><ID>124</ID><Nombre>Kevin</Nombre><Apellido>Mourgos</Apellido_
26 <Empleado><ID>125</ID><Nombre>Julia</Nombre><Apellido>Nayer</Apellido><S_
27 <Empleado><ID>126</ID><Nombre>Irene</Nombre><Apellido>Mikkilineni</Apell_
28 <Empleado><ID>127</ID><Nombre>James</Nombre><Apellido>Landry</Apellido><_
29 <Empleado><ID>128</ID><Nombre>Steven</Nombre><Apellido>Markle</Apellido_

```

Como primer acercamiento a la generación de XML desde Oracle, se construyó una consulta utilizando XMLELEMENT y XMLFOREST sobre la tabla EMPLOYEES. Esta consulta transforma cada fila en un elemento <Empleado> con sus campos básicos. El resultado muestra un documento XML por cada empleado del esquema HR.

Asignatura	Datos del alumno	Fecha
Bases de Datos Avanzadas	Apellidos: Herrera Ramírez	Mayo 17 de 2026
	Nombre: Andres Mateo	

Ilustración 3 Consulta SQL / XML con JOIN entre dos tablas

The screenshot shows the SQL Developer interface. The left pane displays the 'CONNECTIONS' tree with 'Oracle Local' selected. The main editor shows the following SQL query:

```

1 SELECT xmlelement(
2     "Empleado",
3     xmlattributes(e.employee_id AS "id"),
4     xmlforest(
5         e.first_name AS "Nombre",
6         e.last_name AS "Apellido",
7         e.email AS "Correo",
8         e.salary AS "Salario",
9         d.department_name AS "Departamento"
10    )
11 ) AS resultado_xml
12 FROM hr.employees e
13 JOIN hr.departments d
14 ON e.department_id = d.department_id;

```

The bottom pane shows the 'QUERY RESULT' tab with the following output:

Fetches 106 rows in 0.055 seconds

RESULTADO_XML
1 <Empleado id="200"><Nombre>Jennifer</Nombre><Apellido>Whalen</Apellido><Correo>
2 <Empleado id="201"><Nombre>Michael</Nombre><Apellido>Martinez</Apellido><Correo>
3 <Empleado id="202"><Nombre>Pat</Nombre><Apellido>Davis</Apellido><Correo>
4 <Empleado id="114"><Nombre>Den</Nombre><Apellido>Li</Apellido><Correo>DL
5 <Empleado id="115"><Nombre>Alexander</Nombre><Apellido>Khoo</Apellido><C
6 <Empleado id="116"><Nombre>Shelli</Nombre><Apellido>Baida</Apellido><Cor
7 <Empleado id="117"><Nombre>Sigal</Nombre><Apellido>Tobias</Apellido><Cor
8 <Empleado id="118"><Nombre>Guy</Nombre><Apellido>Himuro</Apellido><Corre
9 <Empleado id="119"><Nombre>Karen</Nombre><Apellido>Colmenares</Apellido>
10 <Empleado id="203"><Nombre>Susan</Nombre><Apellido>Jacobs</Apellido><Cor
11 <Empleado id="120"><Nombre>Matthew</Nombre><Apellido>Weiss</Apellido><Co
12 <Empleado id="121"><Nombre>Adam</Nombre><Apellido>Fripp</Apellido><Corre
13 <Empleado id="122"><Nombre>Payam</Nombre><Apellido>Kaufling</Apellido><C
14 <Empleado id="123"><Nombre>Shanta</Nombre><Apellido>Vollman</Apellido><C
15 <Empleado id="124"><Nombre>Kevin</Nombre><Apellido>Mourgos</Apellido><Co
16 <Empleado id="125"><Nombre>Julia</Nombre><Apellido>Nayer</Apellido><Corr
17 <Empleado id="126"><Nombre>Irene</Nombre><Apellido>Mikkilineni</Apellido
18 <Empleado id="127"><Nombre>James</Nombre><Apellido>Landry</Apellido><Cor
19 <Empleado id="128"><Nombre>Steven</Nombre><Apellido>Markle</Apellido><Co
20 <Empleado id="129"><Nombre>Laura</Nombre><Apellido>Bissot</Apellido><Cor
21 <Empleado id="130"><Nombre>Mozhe</Nombre><Apellido>Atkinson</Apellido><C
22 <Empleado id="131"><Nombre>James</Nombre><Apellido>Marlow</Apellido><Cor
23 <Empleado id="132"><Nombre>TJ</Nombre><Apellido>Olson</Apellido><Correo>
24 <Empleado id="133"><Nombre>Jason</Nombre><Apellido>Mallin</Apellido><Cor
25 <Empleado id="134"><Nombre>Michael</Nombre><Apellido>Rogers</Apellido><C

Se amplió la consulta incorporando un JOIN con la tabla DEPARTMENTS, lo que permitió incluir el nombre del departamento en cada elemento XML. Adicionalmente, se utilizó XMLATTRIBUTES para mapear el ID del empleado como atributo del elemento, en concordancia con el esquema XSD definido previamente.

Asignatura	Datos del alumno	Fecha
Bases de Datos Avanzadas	Apellidos: Herrera Ramírez	Mayo 17 de 2026
	Nombre: Andres Mateo	

Ilustración 4 Documento XML completo con XMLAGG y XMLSERIALIZE

```

1 <?xml version='1.0' encoding='UTF-8' ?>
2
3 <Empleados>
4   <Empleado id="200">
5     <Nombre>Jennifer</Nombre>
6     <Apellido>Whalen</Apellido>
7     <Correo>JWHALEN</Correo>
8     <Salario>4400</Salario>
9     <Departamento>Administration</Departamento>
10  </Empleado>
11  <Empleado id="201">
12    <Nombre>Michael</Nombre>
13    <Apellido>Martinez</Apellido>
14    <Correo>MMARTINE</Correo>
15    <Salario>13000</Salario>
16    <Departamento>Marketing</Departamento>
17  </Empleado>
18  <Empleado id="202">
19    <Nombre>Pat</Nombre>
20    <Apellido>Davis</Apellido>
21    <Correo>PDAVIS</Correo>
22    <Salario>6000</Salario>
23    <Departamento>Marketing</Departamento>
24  </Empleado>
25  <Empleado id="114">
26    <Nombre>Den</Nombre>
27    <Apellido>Li</Apellido>
28    <Correo>DLI</Correo>
29    <Salario>11000</Salario>
30    <Departamento>Purchasing</Departamento>
31  </Empleado>
32  <Empleado id="115">
33    <Nombre>Alexander</Nombre>
34    <Apellido>Khoo</Apellido>
35    <Correo>AKHOO</Correo>
36    <Salario>3100</Salario>
37    <Departamento>Purchasing</Departamento>
38  </Empleado>
39  <Empleado id="116">
40    <Nombre>Shelli</Nombre>
41    <Apellido>Baida</Apellido>
42    <Correo>SBAIDA</Correo>

```

El resultado de la consulta fue exportado y almacenado en el archivo andres_herrera.xml. En él se puede apreciar la estructura jerárquica del documento: un elemento raíz <Empleados> que contiene múltiples elementos <Empleado>, cada uno con su atributo id y los cinco campos definidos en el esquema XSD: Nombre, Apellido, Correo, Salario y Departamento.

Asignatura	Datos del alumno	Fecha
Bases de Datos Avanzadas	Apellidos: Herrera Ramírez	Mayo 17 de 2026
	Nombre: Andres Mateo	

Ilustración 5 Validación del XML contra el XSD con herramienta web

Una vez generado el documento XML, se procedió a validarlo contra el esquema XSD utilizando la herramienta web FreeFormatter. El resultado confirmó que el documento cumple con todas las restricciones definidas en el esquema, incluyendo la presencia obligatoria del atributo id y los cinco elementos requeridos por cada empleado.

Ilustración 6 Inserción válida e inválida en Oracle

ID	DOCUMENTO_XML
1	<Empleado id="999"> <Nombre>Andres</Nombre> <Apellido>Herrera</Apellido> <Correo>AHERRERA</Correo>
2	<Empleado id="1000"> <Nombre>Andres</Nombre> </Empleado>
3	<empleados> <empleado id="200"> <nombre>Jennifer</nombre> <apellido>Whalen</apellido>

Asignatura	Datos del alumno	Fecha
Bases de Datos Avanzadas	Apellidos: Herrera Ramírez	Mayo 17 de 2026
	Nombre: Andres Mateo	

Se verificó el contenido de la tabla empleados_xml, confirmando la existencia de tres registros insertados. El primer registro corresponde a una inserción válida que cumple con la estructura definida en el XSD. El segundo registro representa una inserción inválida, ya que contiene únicamente el campo <Nombre>, omitiendo los demás elementos requeridos por el esquema. El tercer registro contiene el documento XML completo generado desde el esquema HR.

Ilustración 7 Consultas XQuery

```

1 (: ===== : )
2 (: CONSULTA 1 - EMPLEADOS CON SALARIO MAYOR A 10000 :)
3 (: ===== : )
4
5 for $e in doc("andres_herrera.xml")/Empleados/Empleado
6 where xs:decimal($e/Salario) > 10000
7 return
8 <EmpleadoAltoSalario>
9   <ID>{ $e/@id }</ID>
10  <Nombre>{ $e/Nombre/text() }</Nombre>
11  <Apellido>{ $e/Apellido/text() }</Apellido>
12  <Salario>{ $e/Salario/text() }</Salario>
13 </EmpleadoAltoSalario>
14
15
16 (: ===== : )
17 (: CONSULTA 2 - EMPLEADOS DEL DEPARTAMENTO SALES :)
18 (: ===== : )
19
20 for $e in doc("andres_herrera.xml")/Empleados/Empleado
21 where $e/Departamento = "Sales"
22 return
23 <EmpleadoSales>
24   <Nombre>{ $e/Nombre/text() }</Nombre>
25   <Apellido>{ $e/Apellido/text() }</Apellido>
26   <Departamento>{ $e/Departamento/text() }</Departamento>
27 </EmpleadoSales>
28
29
30 (: ===== : )
31 (: CONSULTA 3 - MOSTRAR NOMBRE Y CORREO :)
32 (: ===== : )
33
34 for $e in doc("andres_herrera.xml")/Empleados/Empleado
35 return
36 <InformacionEmpleado>
37   <Nombre>{ $e/Nombre/text() }</Nombre>
38   <Correo>{ $e/Correo/text() }</Correo>
39 </InformacionEmpleado>
40
41
42 (: ===== : )

```

Asignatura	Datos del alumno	Fecha
Bases de Datos Avanzadas	Apellidos: Herrera Ramírez	Mayo 17 de 2026
	Nombre: Andres Mateo	

Se desarrollaron cinco consultas XQuery sobre el documento andres_herrera.xml. En la imagen se pueden apreciar las tres primeras: la consulta 1 filtra los empleados con salario mayor a 10.000 retornando un elemento <EmpleadoAltoSalario>; la consulta 2 filtra por departamento Sales; y la consulta 3 recupera el nombre y correo de todos los empleados. Adicionalmente se desarrollaron dos consultas más: la consulta 4 filtra los empleados del departamento IT, y la consulta 5 recupera los empleados con salario entre 5.000 y 10.000.

Conclusiones:

1. *Integración entre bases de datos relacionales y XML:* Oracle ofrece un conjunto robusto de funciones SQL/XML como XMLELEMENT, XMLATTRIBUTES, XMLFOREST y XMLAGG que permiten transformar datos relacionales en documentos XML estructurados de manera eficiente, facilitando la interoperabilidad entre sistemas que requieren este formato para el intercambio de información.
2. *Importancia del esquema XSD como contrato de datos:* La definición de un esquema XSD previo a la generación del documento XML resultó fundamental para garantizar la consistencia y validez de los datos. Al establecer restricciones claras sobre los elementos, atributos y tipos de datos esperados, el XSD actúa como un contrato que asegura la integridad de la información tanto en la generación como en la inserción de documentos.
3. *Validación como mecanismo de control de calidad:* La validación del documento XML contra el esquema XSD, realizada mediante la herramienta FreeFormatter, permitió confirmar que la estructura generada cumple con todas las restricciones definidas. Este proceso evidenció la diferencia entre un documento bien formado y un documento válido, conceptos fundamentales en el manejo de XML en entornos empresariales.